

Music Streaming with Shared Listening Rooms: Backend and Architecture

1. Scale and assumptions

Targets: 20M monthly users, 2M listening at peak, 50M tracks, and 200k rooms at peak with 2 to 5,000 listeners each. The numbers below are estimates from stated assumptions, not measurements.

Quantity	Working	Result
Peak audio egress	2M listeners x 128 kbps	about 256 Gbps
Origin load	5% CDN misses (95% hit rate assumed)	about 13 Gbps
Storage	4 bitrates, 4 min track is about 18 MB, x 50M tracks	about 900 TB
Playback events	2M listeners / about 240 s per track	about 8k per second
Pause in a 5,000 room	5,000 messages x about 200 bytes	about 1 MB, trivial

A CDN is mandatory, because no origin can serve 256 Gbps. Room traffic is tiny next to audio.

2. The core idea

Audio never passes through the room server. Every listener downloads audio from the CDN on their own. The room only shares a small timeline: which track, when it started, whether it is playing, the queue and a version number.

So syncing a room is a clock and position problem, not a streaming problem. Each client works out where the track should be and plays its own copy.

3. Architecture

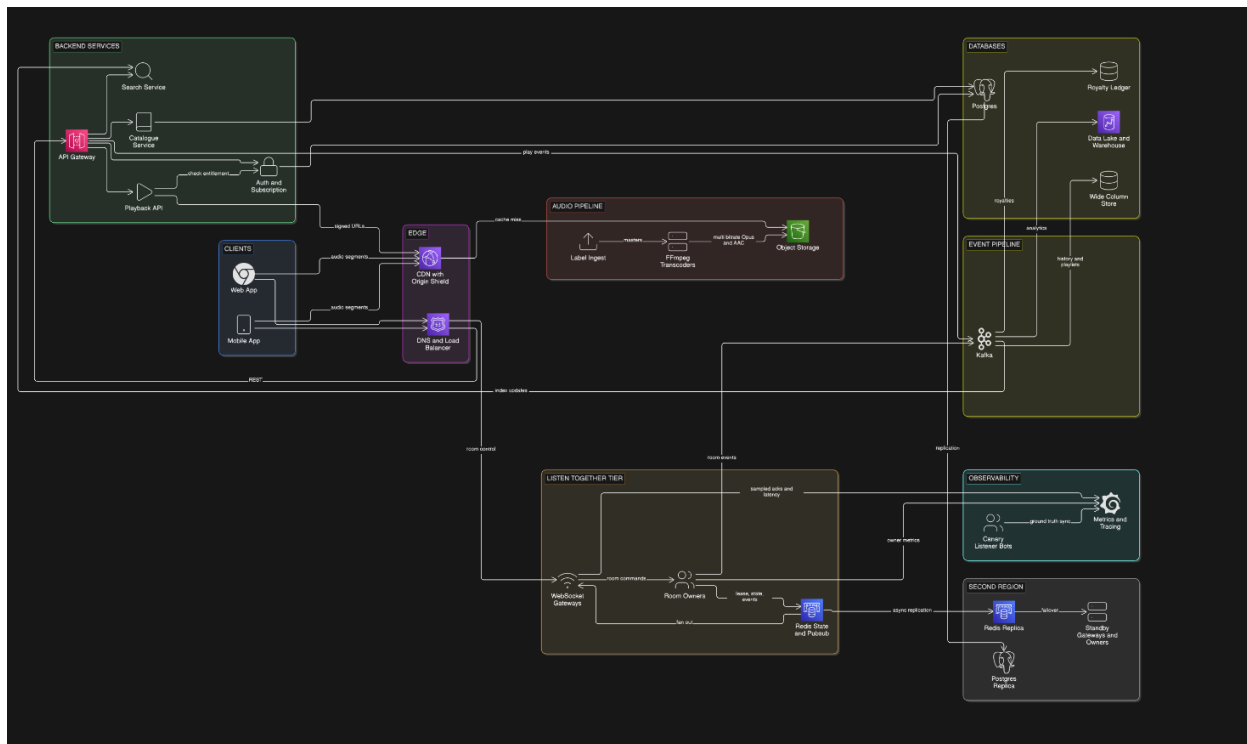


Figure 1: System architecture. Audio flows from the CDN straight to clients, while the room tier only shares small timeline state. . **Find this diagram clearly [here](#)**

The diagram has these parts:

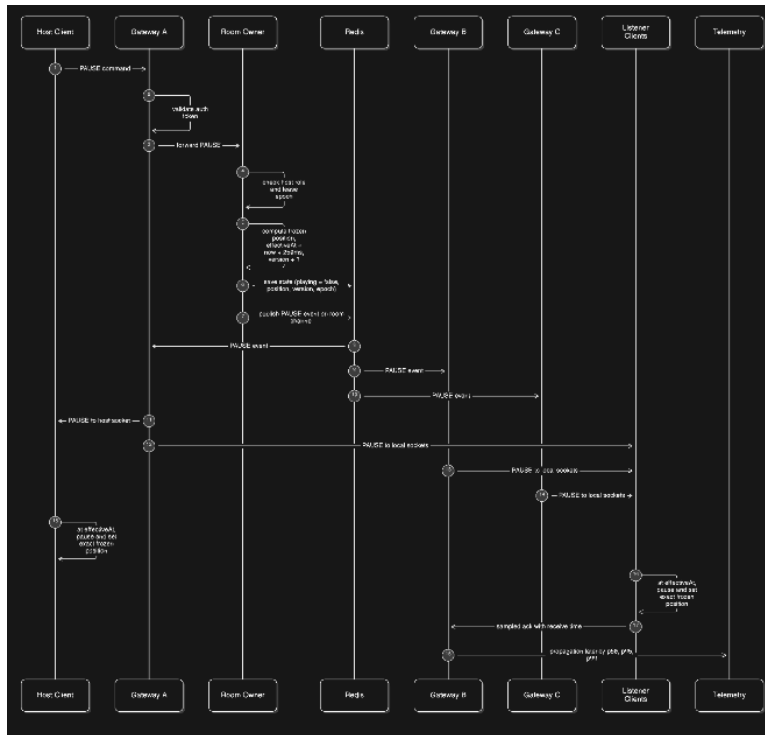
- **Edge:** DNS and a load balancer send users to the nearest healthy region. The CDN, with an origin shield in front of storage, serves all audio.
- **Backend services:** an API gateway in front of auth and subscription, the playback API (checks the subscription and hands out short lived signed URLs), the catalogue and search.
- **Listen Together tier:** WebSocket gateways hold live connections, room owners are the single writer for each room, and Redis holds room state and carries the fan out.
- **Audio pipeline:** labels deliver masters, FFmpeg workers make Opus or AAC at about 24, 96, 160 and 320 kbps in short segments, and they land in object storage. The client picks a bitrate from its speed and buffer (adaptive bitrate) and starts low for a fast start.
- **Catalogue and search:** Postgres holds metadata, and search as you type uses OpenSearch. A new release goes Postgres, then Kafka, then the search index and caches.
- **Data and events:** Kafka feeds listening history (wide column store), the royalty ledger and the data lake. A second region holds replicas.

4. Room sync

- **Clock sync:** each client pings the server, measures the round trip, and estimates its offset from server time, keeping the sample with the lowest round trip.
- **Scheduled actions:** pause, resume and seek carry an effectiveAt time about 250 ms ahead plus the exact position, so everyone acts at the same server instant.

- **Drift correction:** under 50 ms off, do nothing. From 50 to 200 ms, nudge playback speed by about 2 percent. Over 200 ms, jump to the right spot.
- **Track changes, late joiners and slow listeners:** the next track starts at anchor plus the current track's duration, so no message is needed. A late joiner fetches the state, computes the position and starts there. A slow listener drops bitrate first, then seeks to catch up, and the room never waits for them.
- **Queue and votes:** the owner orders all commands. Votes are a set of user ids, so repeats do nothing, and a skip passes at half of connected listeners. Pause and seek are host only.
- **Ownership and failover:** each room has one owner, held by a lease (about 3 second TTL) with an epoch counter that rises when ownership changes. Redis rejects writes with an old epoch (fencing), so a stale owner cannot overwrite newer state. If an owner dies, another node claims the room, loads state from Redis and bumps the epoch. Listeners keep playing, because playback follows the shared clock and not the socket.
- **Fan out:** the owner publishes to a Redis room channel, and each gateway pushes events to its own sockets. A state checks every few seconds, or a version gap, lets a listener that missed a message catch up.

Sequence: host pauses in a room of 5,000 members



Find this diagram clearly [here](#)

Figure 2: Host pause in a room of 5,000 listeners. The owner schedules the pause 250 ms ahead so every listener acts at the same server time.

1. The host client sends PAUSE to its gateway, which checks the auth token and forwards it to the room owner.
2. The owner checks the host role and lease epoch, computes the frozen position, sets effectiveAt to now plus 250 ms, and raises the version.

3. The owner saves state to Redis, so a new owner could take over, then publishes PAUSE on the room channel.
4. Redis delivers it to every gateway holding listeners for the room. Each pushes it to its sockets, including the host's, which also waits for effectiveAt.
5. At effectiveAt every client pauses and sets the exact frozen position. A listener who gets it late pauses at once with the same position.
6. About 1 percent of listeners send an ack with their receive time. Telemetry turns these into propagation latency (p50, p95, p99), which checks the 200 ms target.

5. Reliability and observability

- **When parts struggle:** timeouts, retry limits with jitter and circuit breakers. Degrade in this order: audio first, then room control, then search, then analytics.
- **How we know sync works:** each client reports its playhead minus the expected timeline (p50, p95, p99 by room size and region). Target: 95 percent of listener seconds within 200 ms. We also track host action propagation time, time to first audio and owner failover time.
- **Ground truth:** canary bots join test rooms in each region and measure sync independently, since client reported sync is self reported.

6. Key decisions and rejected alternatives

- **Audio comes from the CDN, the room shares only timeline state**
 1. Why: keeps adaptive bitrate, scales with the CDN, and room traffic stays tiny.
 2. Rejected: one server mixed live stream. No per user bitrate, seconds of latency, and huge egress from our servers.
- **Anchor timeline plus scheduled actions**
 1. Why: position is calculated by each client, and everyone acts at the same server instant.
 2. Rejected: pushing position to every listener constantly. Heavy traffic and still jittery.
- **Redis for room state and fan out**
 1. Why: fast, and a good fit for small, hot, temporary data.
 2. Rejected: a relational database write on every action. Too slow for 200 ms sync.
- **Kafka for events**
 1. Why: a durable log many consumers can read (history, royalties, analytics), with idempotency keys(to avoid duplication) so repeats are harmless.
 2. Rejected: a plain queue. One consumer and no replay.

7. Failure scenarios

- **Room owner node dies**
 - o Impact: audio continues, controls are delayed a few seconds.
 - o Handling: the lease expires, another node claims the room, loads state from Redis and bumps the epoch.
- **WebSocket gateway dies or is being deployed**
 - o Impact: a brief reconnect, audio continues.
 - o Handling: clients reconnect with random delays (jitter) so there is no stampede, then fetch the current state.

- **Whole region goes down**
 - o Impact: rooms reconnect, audio keeps coming from the CDN.
 - o Handling: re home rooms in the second region under a new epoch. At worst a few seconds of room state is lost.
- **Host disconnects**
 - o Impact: no host for a moment.
 - o Handling: after a short timeout the host role passes to another listener.
- **Vote spam or one hot room**
 - o Impact: the spammer is throttled, other rooms are unaffected.
 - o Handling: per user rate limits, per room isolation, and votes that count once per user.
- **Kafka lag or outage**
 - o Impact: none for playback.
 - o Handling: buffer events and retry. Royalty events are never dropped.
- **A listener with a bad clock or slow network**
 - o Impact: that one listener drifts.
 - o Handling: drift correction, a lower bitrate, and catching up by seeking.

8. What I would build first with a team of three, and what to postpone

Build first:

- One region on managed services.
- Audio: 2 or 3 bitrates, one CDN, signed URLs, adaptive bitrate on the client.
- Catalogue: Postgres plus a managed search service.
- Room sync: gateway and room service combined to just a room service, Redis for room state, the anchor timeline with clock sync, scheduled pause, queue and skip votes.
- Basics: metrics for sync error, time to first audio, and a minimal event pipeline to record important events.
- Split of work: one person on audio, catalogue and search. One on room sync and the client. One on infrastructure, events and observability.

Postpone:

- Multi region active active or even active standby. One region plus a replica is enough to start.
- Multi CDN. One is fine until traffic grows.
- Recommendations and personalisation.
- Custom DRM. Buy it first.
- Offline downloads.
- Collaborative playlist editing.
- Social features like follow and in room chat.